SELF_HEAL.md — Sakum Self-Learning & Self-Healing Registry

ONE FILE that registers the whole self-healing mechanism: how the engine

checks the internet, how it fixes code, every patch it has applied (with a

comment on what was broken and what the fix was), and the single rolling

survivability metric. This is the consolidated record; memory.md stays the

append-only raw ledger, this file is the human-readable single source.

Last regenerated: cycle 1784360162

==========

1. HOW THE ENGINE ACTUALLY WORKS (is it live, or static?)

==========

It is REAL CODE, but it is DORMANT until triggered. There is no infinite background process burning CPU right now — the loop only runs when kicked:

TRIGGERS (tools/sakum_bot.sh, learn.md §Triggers):
- launchd timer -> com.sakum.bot.plist (ALWAYS-ON job, loaded)
- HTTP webhook -> POST /update (tools/serve.sh native x86-64)
- WebSocket frame -> ws://127.0.0.1:8765
- manual -> bash tools/sakum_bot.sh --once

THE LOOP (one cycle):
0. git_self_upgrade.sh -> rebase upstream Sakum Lang commits, recompile
1. read doctrine -> SAKUM_LANG.md + learn.md + memory.md
2. fetch_updates.sh -> webfetch trusted PL sources (LLVM / Rust / WASM-spec / GCC GitHub releases), emit "SIGNAL " lines. ONLY keywords SIMD|AVX|NEON|RVV|WASM|quantum| memory.safe|post.quantum|crypto|numeric| overflow|bounds are kept; hype dropped.
3. decide + GENERATE -> gen_lib.sh emits a REAL compilable assembly/sakum_lib_.s (NOT a stub). If no upstream signal, self-directed learning pulls the next topic off a roadmap queue (simd/wasm/quantum/crypto/...).
4. recompile gate -> gcc -arch x86_64 over every assembly/sakum_*.s (ARM/RISC-V variants skipped; separate ISA).
5. SELF-HEAL -> if compile FAILS: roll back THIS cycle's generated .s + patch_.json, write a `mistake` line to memory.md + the binary-hash ledger, exit 2. launchd KeepAlive relaunches.
6. remember -> survive += 1, patches_applied += 1, append `learned` line, update last_cycle/last_check.

INTERNET CHECK: YES — fetch_updates.sh hits GitHub release APIs every pulse. It does NOT apply random internet code; it only extracts topic SIGNALS and uses them to decide which LOCAL library to generate/extend. The bot never pulls untrusted code into the core; every generated artifact is raw assembly written by gen_lib.sh.

SELF-LEARNING: YES — survives/fails are counted, mistakes are logged with root cause, and bad patches are rolled back so the same mistake is not repeated. The `survive` counter only rises on a clean compile+run.

CURRENT STATE: static read right now. com.sakum.bot is loaded but no process is live and serve.sh is not up, so the 80% below is computed from memory.md at page-load, not a running loop. Run `bash tools/sakum_bot.sh --once` (or start serve.sh) to make it live.

==========

2. SURVIVABILITY (single metric, computed from memory.md)

==========

formula: survivability = survive / (survive + mistakes) * 100
where:
survive = rolling clean compile+run counter (memory.md: survive:)
mistakes = count of real `mistake <ts>:` ledger entries in memory.md

CURRENT:
survive = 48
mistakes = 12 (10 historical + 2 fixed this cycle)
score = 48 / (48 + 12) = 80 %

NOTE: the dashboard (site/app/site.js) was buggy — it counted every
substring "mistake" (incl. the "## mistakes" header and prose), which
inflated the denominator. Fixed to count only `^mistake` ledger lines
(site/app/site.js:220). Score is now honest: 80%.

The "Survivability" panel reads site/memory.md live via fetch(); it is NOT
hardcoded. Reload the served page to see it update.

==========

3. PATCH REGISTRY — every fix, with what-it-fixed comment

==========

--- PATCH H1 — assembly/sakum_pipe.s (cycle 1784360162) ----------
PROBLEM:
The file redefined SYS_READ/SYS_WRITE/... as assembler `= <n>` equ's, but
assembly/platform.inc already does `#define SYS_READ 0x2000003` (under
PLAT_MACOS). The C preprocessor expanded the symbol name BEFORE the
assembler saw it, turning the line into:
`0x2000003 = 0x2000000 + 3`
which is an invalid statement -> "unexpected token at start of statement".
FIX (comment: guard each equ so it only defines when platform.inc has not):
Wrapped every SYS_* = ... in `#ifndef SYS_*` / `#endif`. platform.inc's
value wins; the file supplies the value only when absent.
RESULT: pipe.s now compiles clean under gcc -arch x86_64 -x assembler-with-cpp.

--- PATCH H2 — assembly/sakum_sniff.s (cycle 1784360162) ----------
PROBLEM (two bugs):
(a) Labels `db0..db3` / `ob0..ob3` collided with GAS's `db` (define-byte)
directive. `[rip + db0]` failed with "invalid base+index expression"
because the assembler lexed `db0` as directive `db` + operand `0`.
(b) After the RODATA `.asciz` string block there was NO `TEXT_SECTION`
switch, so `main` + all following code was emitted into __cstring.
The linker then raised "4 byte relocation not fully within bounds of
atom" and warned symbols were "located within another string".
FIX (comments inline):
(a) Renamed db0..3 -> oct_d0..3 and ob0..3 -> oct_s0..3 everywhere

(declarations + all [rip + ...] references).
(b) Added `TEXT_SECTION` immediately before `CDECL(main):` so code lives
in __text, not __cstring.
RESULT: sniff.s now compiles AND links clean; runs as a packet sniffer.

--- PATCH H3 — site/app/site.js:220 (cycle 1784360162) ----------
PROBLEM:
Dashboard survivability counted `mistake` substring occurrences (incl.
"## mistakes" header + prose), not real ledger entries -> wrong %.
FIX:
Changed `text.match(/mistake/g)` to `text.match(/^mistake\s/gm)` so only
actual `mistake <ts>:` ledger lines count. Score is now honest (80%).

--- PATCH H4 — site/memory.md (cycle 1784360162) ----------
PROBLEM:
Dashboard reads site/memory.md, which was STALE (survive: 35,
last_cycle: 1784205501, empty mistakes). The root memory.md had been
updated to survive: 48 but the SITE copy was not, so the page showed
0% / "no entries yet".
FIX:
Synced site/memory.md: survive: 48, last_cycle: 1784360162, added the 2
real mistake entries + the learned entry. Page now reflects reality.

--- HISTORICAL MISTAKES (already in ledger, status: fixed) ----------
These predate this cycle and are the other 10 `mistake` entries:
* Undefined symbols for architecture x86_64 (link error, fixed)
* sakum_pipe.s: "token is not a valid binary operator in a preprocessor
subexpression" at lines 815/850/857/859/865/866 (cpp #if bug, fixed)
Each was rolled back at its time and re-fixed; they remain in the ledger so
the engine does not repeat them.

==========
4. CONSOLIDATED STATUS (one glance)
==========

engine mode ..... DORMANT (loaded, not running) — start with sakum_bot.sh
internet check .... fetch_updates.sh -> GitHub LLVM/Rust/WASM/GCC releases
self-heal ..... rollback generated .s + patch json on compile fail
survivability ..... 80 % (survive 48 / mistakes 12)
last cycle 1784360162
patches this cycle 4 (pipe, sniff, site.js, site memory.md)
dashboard ..... http://127.0.0.1:8099/index.html (live fetch of memory.md)

TO MAKE IT LIVE:
bash tools/sakum_bot.sh --once # one pulse
or always-on:
cp tools/com.sakum.bot.plist ~/Library/LaunchAgents/
launchctl load ~/Library/LaunchAgents/com.sakum.bot.plist
bash tools/serve.sh 8080 & # webhook + timer pulse endpoint

==========

5. CROSS-PLATFORM BUILD (all OS x all ISA) + NEXT-SURVIVAL-CODE SUGGESTER

==========

Directives honored: every artifact must be raw machine code for
Windows / macOS / Linux x { x86-64, arm64, riscv64, arm32 }.

VERIFIED TO BUILD (this host, cross-toolchains installed via brew):
x86-64 macOS -> gcc -arch x86_64 (all sakum_*.s)*
x86-64 linux -> gcc -m64 (Makefile X86_64_TARGETS)
arm64 macOS -> gcc -arch arm64 (tracker_arm64, sys_arm64)
arm64 bare -> aarch64-elf-gcc -nostdlib (tracker_arm64_neon)
riscv64 bare -> riscv64-elf-gcc -march=rv64gcv (sys_riscv64, trackers)
arm32 bare -> arm-none-eabi-gcc -march=armv7-a (tracker_arm32)
(Windows x86-64 subset builds with mingw-w64 per Makefile `win` target.)

PATCH H5 - assembly/sakum_sys_riscv64.s (this cycle) ----------
PROBLEM: RVV compare `vmsle.vf v1, v0, zero` used integer reg `zero`
as a float operand -> "unrecognized opcode".
FIX: build a zero vector (vmv.v.x v1, zero) and use `vmsle.vv v1, v0, v1`.
RESULT: sys_riscv64 now builds under riscv64-elf-gcc.

PATCH H6 - assembly/sakum_lib_survive.s (NEW library fn) ----------
WHAT: machine-level "next survival code" core. Scans a source buffer,
finds the NEXT indentation depth, and returns a per-ISA/OS template
id (generic / bounds / stack-align). Pure byte scan -> ISA-agnostic,
runs on every platform. Built via `make lib_survive`.
WHY: it is the engine that powers the suggester below.

PATCH H7 - tools/sakum_suggest.sh (NEW advisor, wired into bot cycle) ----
WHAT: the self-healing advisor. For a target it:
1. calls sakum_lib_survive to find next indent + template id
2. generates the SAME bounds-check guard in native syntax for
x86-64(mac/linux), arm64, riscv64, arm32, + Sakum source
3. VERIFIES each candidate through the GATE:
- compiler gate : assembles on EVERY target toolchain (no fail = safe)
- lexer gate : Sakum interpreter lexes the .sak candidate
- Sakum AI note : prints WHAT / HOW / WHY the snippet is safe
4. only SUGGESTS snippets that pass all gates; logs to memory.md.
If any candidate fails, the engine self-heals (does NOT suggest).
RESULT (live run): lexer PASS, compiler gate PASS=5 FAIL=0.
Now invoked automatically at the end of every sakum_bot.sh pulse.

The suggester is the "always check + suggest next survival code" mechanism:
it never proposes an edit that the compiler/lexer would reject on ANY
platform, and the Sakum internal AI explains each suggestion in plain words.

==========

6. SANSKRIT -> HINGLISH ONLY (keyword policy)

==========

Directive: the language accepts ONLY Sanskrit (Devanagari) and its Hinglish (romanized) spelling. Pure-English keywords are NOT part of the language.

ENFORCED IN site/app/sakum.js:
- KW map keeps only Devanagari + Hinglish (e.g. नाम/naam, यदि/yadi, लेख/lek, चर/char, सूत्र/sutra, वापस/vapsa, ब्रम्ह/brahma, परीक्षा/pariksha, और/aur).
- RESERVED_ENGLISH set makes the LEXER REJECT let/fn/if/else/while/for/return/print/class/and/or (throws "English keyword not allowed").
- Math/utility BUILTINS (vec, sin, cos, sqrt, len, ...) stay English by design (universal, not control keywords).

PATCH H8 - grammar drift fixed (this cycle) ----------
The shipped examples used Sanskrit+Hinglish the OLD interpreter didn't parse (चर, वापस, मुद्रण, सूत्र, ब्रम्ह.learn, परीक्षा, and/or, [], classes, single-quoted strings, grouped '(', XOR '^', जबतक/लंबाई). Fixed:
* added missing keywords: चर, वापस, मुद्रण, सूत्र, ब्रम्ह, परीक्षा, और/अथवा, जबतक, लंबाई, वर्ग (class)
* member calls ब्रम्ह.learn / नाडी.socket (id.id(args))
* paren-free print (मुद्रण "a" x), optional ';', vec decl वेक्टर D[8], class decl वर्ग Name { ... }, empty array [], single-quoted strings, grouped '(expr)', XOR '^', logical और/अथवा
RESULT: all 14 example .sak files now PARSE; English keywords rejected; Sanskrit/Hinglish accepted. lib examples also RUN.
Updated SAKUM_LANG.md keyword table to Sanskrit + Hinglish only.